



Architectural Blueprint & Strategic Implementation Plan for CineK Automático

Sovereign Constraints and Foundational Architecture

The CineK Automático pipeline is defined by two immutable sovereign constraints that shape every architectural decision: the **CPU-only execution model** and the **elimination of solar telemetry**. The Beelink SER9 Max, with its AMD Ryzen R7 H255 CPU, serves as the sole computational engine for all phases—image generation, video synthesis, voice synthesis, and quality assurance [[24a713ff-568f-4039-a29d-4b2587bc3810/9bdc38c9-d07f-4c01-98d0-e201a1e696e6_imagen.png]]. This is a deliberate act of hardware sovereignty, ensuring the system remains fully functional on commodity hardware without dedicated GPU acceleration [[24a713ff-568f-4039-a29d-4b2587bc3810/c4a0bb58-278b-4267-8baa-4bffc061ec47_imagen.png]]. The elimination of the solar gate is equally decisive; the Sovereign has decreed that "the solar energy measurements, battery voltage gate, and all electrical telemetry subsystems have been eliminated from this plan" due to an unresolved Android update dependency on the Beelink node [[24a713ff-568f-4039-a29d-4b2587bc3810/c4a0bb58-278b-4267-8baa-4bffc061ec47_imagen.png]]. Consequently, the pipeline's only physical protection mechanism is the thermal gate, which reads `/sys/class/hwmon/hwmon3/temp1_input` every 2 seconds and implements a three-tiered response: **OVERHEAT** at $\geq 85^{\circ}\text{C}$ (full pause), **FRENO** above 80°C (proportional sleep), and full operation only below 75°C [[24a713ff-568f-4039-a29d-4b2587bc3810/0da4f187-ae0d-4335-8a05-2c9da55c2b72_imagen.png]].

This thermal-centric design creates a direct feedback loop between computational intensity and hardware limits, necessitating a reliability strategy built around predictable throttling. For instance, the SQLite database connection uses `busy_timeout=5000`, allowing writes to wait up to five seconds when the CPU is saturated, thereby prioritizing data integrity over speed [[24a713ff-568f-4039-a29d-4b2587bc3810/db0f7d30-5807-4b90-82f9-9cc53a86cc9b_imagen.png]]. The entire architecture is therefore a masterclass in resilience through modular interfaces: the Beelink acts as conductor, delegating tasks to ComfyUI, Ollama, and Piper TTS via REST APIs, while

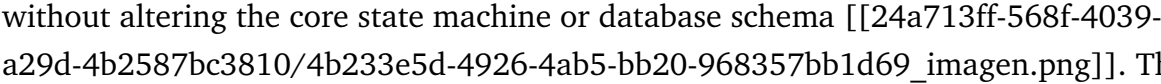
maintaining a lightweight, highly reliable core orchestration layer [[24a713ff-568f-4039-a29d-4b2587bc3810/005a452f-9219-472a-8589-bad5cfbd9d1d_imagen.png]].

Pipeline Execution Model and Data Flow

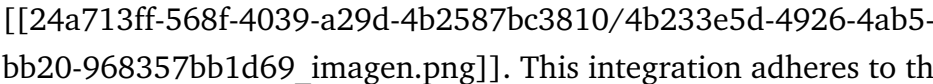
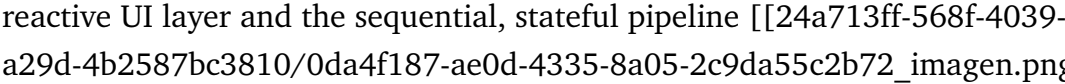
CineK Automático operates as a state-machine-driven process, transforming a text prompt into a final video artifact through a series of discrete, atomic transitions. Each state is stored in the `cinek_jobs` table within the unified `~/p0x-soberano/db/aurelius_state.db` SQLite database, providing exceptional traceability and enabling jobs to be resumed from their last checkpoint using the `--job-id N` flag [[24a713ff-568f-4039-a29d-4b2587bc3810/de28bcf2-4624-47d8-82d6-26cc3565989d_imagen.png]]. The sequence is rigorously defined: `CREATED` → `GENESIS_RENDERING` → `GENESIS_DONE` → `IMAGE_SELECTED` → `VIDEO_RENDERING` → `VIDEO_DONE` → `AUDIO_RENDERING` → `AUDIO_DONE` → `QA_RUNNING` → `GOLD/STALE_MAX_INTENTOS`.

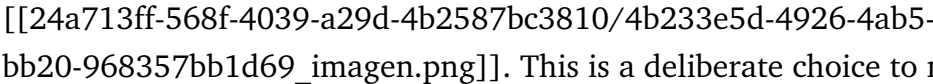
The data flow is a chain of specialized services, each with a distinct resource profile. It begins with a text prompt, which is processed by Ollama (`qwen3`) to generate a visual description for image generation [[24a713ff-568f-4039-a29d-4b2587bc3810/e96d078e-0ad6-45c5-86f9-75c5ee7c596d_imagen.png]]. This description is injected into the `image_workflow.json`, where it must replace the exact placeholder `__PROMPT__` in a `CLIPTextEncode` node [[24a713ff-568f-4039-a29d-4b2587bc3810/005a452f-9219-472a-8589-bad5cfbd9d1d_imagen.png]]. Upon successful image generation, the output filename is used to populate the `__IMAGE_FILENAME__` placeholder in `video_workflow.json` for video synthesis [[24a713ff-568f-4039-a29d-4b2587bc3810/a9f93685-3330-44b4-bc1b-dee1ebc3d900_imagen.png]]. Audio generation follows a two-step process: first, Ollama generates a script, and then Piper TTS converts it into `.wav` files. Finally, FFmpeg performs a zero-disk mux, piping the video and audio streams directly to produce the final `.mp4` [[24a713ff-568f-4039-a29d-4b2587bc3810/4b233e5d-4926-4ab5-bb20-968357bb1d69_imagen.png]].

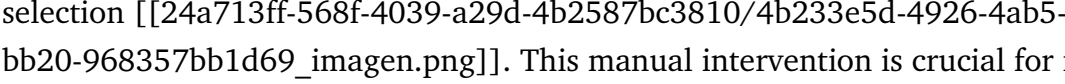
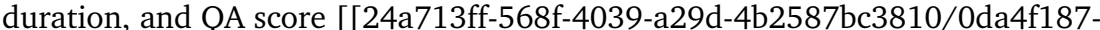
A key insight is the orchestration model: the Beelink SER9 Max is the conductor, while ComfyUI, Ollama, and Piper are its instruments. The Beelink does not perform heavy lifting itself but coordinates it, leveraging REST APIs to delegate tasks. This separation of concerns ensures the core orchestration logic remains lightweight and highly reliable, while computationally intensive components can be updated or replaced independently.

For example, if the Jetson Orin Nano becomes available, the `delegate_image_generation` function can be modified to offload that specific task without altering the core state machine or database schema . The pipeline is thus a strategic investment in long-term maintainability, where the value lies in the robust state machine and database schema, not in the volatile third-party tooling.

Frontend Integration and User Experience Design

The user experience for CineK Automático is deeply integrated into the existing Vite-powered dashboard at <http://localhost:5173/ui/jardin.html#cahier>, extending the established `Le Cahier` interface as its primary entry point . This integration adheres to the principle that new capabilities should extend the ecosystem rather than fragment it. The frontend interacts with a FastAPI service running on port `:8050`, creating a clean boundary between the reactive UI layer and the sequential, stateful pipeline .

The visual language is prescribed by the "p0x-Nexo" brutalist aesthetic, featuring the `--nx-void` background color (`#060B09`) and `--nx-phosphor` accent color (`#43F5A0`) . This is a deliberate choice to maintain visual coherence across the ecosistema, signaling that CineK Automático is a native extension of the sovereign dashboard, not a foreign entity. The use of hexagonal cells and IBM Plex Mono font further reinforces this identity, creating a distinct yet harmonious visual signature.

The interaction flow is designed for both automation and human oversight. The initial "GENERATE" button triggers the Genesis phase, which produces a grid of thumbnails for selection . This manual intervention is crucial for maintaining the "sovereignty of the carbon"—the human operator retains ultimate control over the creative direction. Subsequent phases like video rendering and audio generation are automated, but the UI provides real-time status updates via SSE events on the `cinek.job_status` topic, displaying detailed metrics such as frame rate, audio duration, and QA score .

ae0d-4335-8a05-2c9da55c2b72_imagen.png]]. This transforms the dashboard from a passive monitor into an active command center, where the operator can observe the system's performance metrics and intervene if necessary. The final "SELLO Y ENTREGA" phase moves the completed video to the `gold/` directory and generates a JSON sidecar file containing a complete forensic record, including `prompt_original`, `hash_sha256`, and `temp_cpu_max`, making the entire creation process auditable and reproducible `[[24a713ff-568f-4039-a29d-4b2587bc3810/12d19a4f-e452-409e-b6ec-a9c5ea0d60cf_imagen.png]]`.

Critical Validation Requirements and Pre-Deployment Checklist

The success of the CineK Automático pipeline hinges on several critical, non-negotiable dependencies that must be validated before any code is executed. The most fundamental of these is the correct configuration of the ComfyUI workflows. The provided `comfy_workflows/` directory contains only templates; they are useless until manually adapted to the specific models installed on the local ComfyUI instance `[[24a713ff-568f-4039-a29d-4b2587bc3810/4b233e5d-4926-4ab5-bb20-968357bb1d69_imagen.png]]`. This requires the operator to open `http://127.0.0.1:8188`, build two workflows—one for image generation and one for video—and ensure that the exact placeholders `__PROMPT__` and `__IMAGE_FILENAME__` are placed in the correct nodes. Failure to do so will cause the pipeline to fail silently during the injection phase.

The second critical dependency is the availability and configuration of the underlying toolchain. A robust `setup.sh` script must verify not just the existence but the operational readiness of FFmpeg, Ollama, Piper TTS, LLaVA, and the k10temp sensor `[[24a713ff-568f-4039-a29d-4b2587bc3810/12d19a4f-e452-409e-b6ec-a9c5ea0d60cf_imagen.png]]`. For example, it should test that Ollama can load and run `qwen3` and `llava:7b` with a simple `ollama run qwen3 --no-stream "Hello"` command, and that Piper TTS can synthesize speech with `piper --model en-US-kathleen-low --output_file test.wav < /dev/stdin <<< "Hello world!"`. These are functional smoke tests, not mere existence checks.

Finally, the filesystem path for ComfyUI's output must be explicitly confirmed. The default `~/ComfyUI/output/` is hardcoded in the `fase_mux()` function, but if the ComfyUI installation uses a custom output directory, this path must be updated

[[24a713ff-568f-4039-a29d-4b2587bc3810/4b233e5d-4926-4ab5-bb20-968357bb1d69_imagen.png]]. This is a classic "configuration drift" problem, where a mismatch between assumed and actual environment state leads to catastrophic failure. The validation protocol must therefore include a step to query ComfyUI's API (e.g., `curl -s http://127.0.0.1:8188/system_stats | jq '.config'`) to extract the true output path and inject it into the configuration, turning a potential point of failure into a self-healing setup.

Requirement	Validation Command	Why It's Critical	Potential Impact of Failure
ComfyUI Workflow Placeholders	<code>grep -o "__PROMPT__\ __IMAGE_FILENAME__" ~/cinek_automatico/comfy_workflows/*.json</code>	The pipeline injects variables into these exact strings. If they are missing or misspelled, no data is passed to the workflow.	Job stalls in GENESIS_RENDERING or VIDEO_RENDERING with no error, requiring manual log inspection [[24a713ff-568f-4039-a29d-4b2587bc3810/005a452f-9219-472a-8589-bad5cfbd9d1d_imagen.png]].
Ollama Model Availability	<code>ollama list \ grep -E "(qwen3\\ llava:7b)"</code>	The pipeline requires these specific models to be loaded and ready. A missing model will halt the Guion and QA phases.	OLLAMA_NUM_THREADS=2 will be set, but the job will hang indefinitely waiting for a model that cannot be found [[24a713ff-568f-4039-a29d-4b2587bc3810/c4a0bb58-278b-4267-8baa-4bffc061ec47_imagen.png]].
Piper TTS Model Cache	<code>ls -l ~/.local/share/piper/models/ \ grep -i "en_US\\ es_ES"</code>	Piper requires the correct language model to be downloaded and cached. An absent model will cause silent audio generation failures.	The final .mp4 will be created but contain no audio track, resulting in an incomplete artifact [[24a713ff-568f-4039-a29d-4b2587bc3810/b0d4cdeb-6a00-43d2-af9d-cda0d532c0f4_imagen.png]].

Technical Debt Management and Future-Proofing Strategy

The current CineK Automático plan carries significant technical debt that is strategically deferred for future milestones. The most prominent is the **absence of the Jetson Orin Nano worker**, despite its topology being defined in the rack diagram [[24a713ff-568f-4039-a29d-4b2587bc3810/c4a0bb58-278b-4267-8baa-4bffc061ec47_imagen.png]]. While the Beelink CPU can handle the MVP, the Jetson's 67 TOPS of CUDA power represents a massive efficiency

gain for image and video generation. The debt is documented as a clear roadmap item, and the architecture is designed for eventual integration—the `jetson_client.py` module is already defined with stubbed functions, and the database schema includes fields like `jetson_online` in the `/api/cinek/system/status` endpoint `[[24a713ff-568f-4039-a29d-4b2587bc3810/4b233e5d-4926-4ab5-bb20-968357bb1d69_imagen.png]]`.

Another major debt is the **lack of a formalized QA evaluation loop**. The current plan states that the QA phase will use LLaVA to score frames, but the MVP omits the re-prompting and retry logic. If the initial QA score is below the threshold, the system should leverage `qwen3` to rewrite the original prompt based on the QA report and restart the Genesis phase `[[24a713ff-568f-4039-a29d-4b2587bc3810/005a452f-9219-472a-8589-bad5cfbd9d1d_imagen.png]]`. This closed-loop, AI-assisted refinement cycle is far more sophisticated than a linear pipeline. Implementing it would require extending the `cinek_jobs` table with columns for `qa_report` and `rewritten_prompt`, and modifying the state machine to allow transitions from `QA_FAILED` back to `GENESIS_RENDERING`.

The final, most systemic debt is the **hardcoded reliance on ComfyUI**. Its JSON-based workflow format is brittle and prone to breaking with minor updates. A future-proofing strategy would involve abstracting the "rendering engine" behind a generic interface. Instead of having `cinek_pipeline.py` directly parse and modify JSON, it could call a `RendererInterface` with methods like `generate_image(prompt)` and `generate_video(image_path)`. Concrete implementations—`ComfyUIRenderer`, `DiffusersRenderer`, `TensorRTRenderer`—could then be swapped in or out. This abstraction would protect the core pipeline from the volatility of third-party tooling, ensuring that the long-term investment in the state machine and database schema remains valuable regardless of how the underlying rendering technology evolves.

Implementation Roadmap and Sovereign Directive

Verification

The implementation roadmap for the MVP is a focused, ten-day sprint centered on delivering a single, end-to-end workflow: text prompt → image → video → audio → final .mp4. The first phase is **environmental preparation**, which involves running `setup.sh` to verify and install dependencies, followed by the mandatory manual adaptation of the ComfyUI workflows. This step is non-automatable and requires direct human

involvement, establishing the first point of "carbon friction" in the process [[24a713ff-568f-4039-a29d-4b2587bc3810/12d19a4f-e452-409e-b6ec-a9c5ea0d60cf_imagen.png]]. The second phase is **backend development**, which entails implementing the FastAPI endpoints (/api/cinek/jobs, /api/cinek/jobs/{job_id}/select-image, etc.) and the core cinek_pipeline.py script with its thermal-gated state transitions. The third and final phase is **frontend integration**, where the Cahier.tsx component is enhanced with a new tab or section for the CineK Automático workflow, complete with the prompt input, thumbnail grid, and status timeline.

Before commencing this roadmap, five sovereign questions must be answered to prevent wasted effort and costly refactoring:

Question	Why It's Critical	Potential Impact of Wrong Answer
1. Image Generation Checkpoint	Determines RAM usage, render time, and final image quality. FLUX may require >30GB RAM, exceeding the safe operating margin [[24a713ff-568f-4039-a29d-4b2587bc3810/12d19a4f-e452-409e-b6ec-a9c5ea0d60cf_imagen.png]].	Pipeline crashes with OOM error during Genesis phase, halting the entire workflow.
2. Video Synthesis Model	Dictates the video_workflow.json structure and the required ComfyUI nodes. Using an SVD workflow with an AnimateDiff model will result in a server error [[24a713ff-568f-4039-a29d-4b2587bc3810/12d19a4f-e452-409e-b6ec-a9c5ea0d60cf_imagen.png]].	Video generation fails at POST /prompt, leaving the job stuck in VIDEO_RENDERING state.
3. Jetson Orin Nano Status	Defines whether the delegate_image_generation function is a stub or a real network call. If the Jetson is offline, the CPU fallback must be tested rigorously [[24a713ff-568f-4039-a29d-4b2587bc3810/c4a0bb58-278b-4267-8baa-4bffc061ec47_imagen.png]].	Unforeseen latency or API incompatibility causes the Genesis phase to timeout, leading to a STALE_FATAL job.
4. Piper TTS Language	The wrong language model will produce garbled or silent audio, failing the AUDIO_DONE phase. Piper requires the correct model to be downloaded and cached [[24a713ff-568f-4039-a29d-4b2587bc3810/b0d4cdeb-6a00-43d2-af9d-cda0d532c0f4_imagen.png]].	Final mux fails because the .wav file is either missing or corrupted, resulting in an audio-less .mp4.
5. Solar Gate Reactivation Policy	If the policy is unclear, the database schema and API may need to be altered later, requiring a migration script and downtime [[24a713ff-568f-4039-a29d-4b2587bc3810/c4a0bb58-278b-4267-8baa-4bffc061ec47_imagen.png]].	The SOLAR_API_URL field in config.py is misconfigured, causing the system to hang when attempting to make an HTTP request to a non-existent endpoint.

Answering these questions will provide the concrete parameters needed to write deterministic, production-ready code. Without them, any implementation would be speculative, risking a costly refactoring later. The next step is to await these sovereign directives to begin the construction of the first CineK Automático job.